

Plugins

Plugins are readily available software that has already been released by third parties and have given approval to the creators of Metasploit to integrate their software inside the framework. These can represent commercial products that have a **Community Edition** for free use but with limited functionality, or they can be individual projects developed by individual people.

The use of plugins makes a pentester's life even easier, bringing the functionality of well-known software into the **msfconsole** or Metasploit Pro environments. Whereas before, we needed to cycle between different software to import and export results, setting options and parameters over and over again, now, with the use of plugins, everything is automatically documented by msfconsole into the database we are using and hosts, services and vulnerabilities are made available at-a-glance for the user. **Plugins** work directly with the API and can be used to manipulate the entire framework. They can be useful for automating repetitive tasks, adding new commands to the **msfconsole**, and extending the already powerful framework.

Using Plugins

To start using a plugin, we will need to ensure it is installed in the correct directory on our machine. Navigating to **/usr/share/metasploit-framework/plugins**, which is the default directory for every new installation of **msfconsole**, should show us which plugins we have to our availability:

Plugins							
dbcyph0n@htb[/htb]\$ ls /usr/share/metasploit-framework/plugins							
aggregator.rb	beholder.rb	event_tester.rb	komand.rb	msfd.rb	nexpose.rb	request.rb	session.rb
alias.rb	db_credcollect.rb	ffautoregen.rb	lab.rb	msgRPC.rb	openvas.rb	rssfeed.rb	session.rb
auto_add_route.rb	db_tracker.rb	ips_filter.rb	libnotify.rb	nessus.rb	pcap_log.rb	sample.rb	socket.rb

If the plugin is found here, we can fire it up inside **msfconsole** and will be met with the greeting output for that specific plugin, signaling that it was successfully loaded in and is now ready to use:

MSF - Load Nessus

Plugins	
msf6 > load nessus	
[*] Nessus Bridge for Metasploit [*] Type nessus_help for a command listing [*] Successfully loaded Plugin: Nessus	
msf6 > nessus_help	
Command	Help Text
-----	-----
Generic Commands	
-----	-----
nessus_connect	Connect to a Nessus server
nessus_logout	Logout from the Nessus server
nessus_login	Login into the connected Nessus server with a different username and
<SNIP>	
nessus_user_del	Delete a Nessus User
nessus_user_help	Display Nessus User Command

```

nessus_user_passwd      Change Nessus Users Password

Policy Commands
-----
nessus_policy_list      List all policies
nessus_policy_del        Delete a policy

```

If the plugin is not installed correctly, we will receive the following error upon trying to load it.

```

                                     Plugins
-----
msf6 > load Plugin_That_Does_Not_Exist

[-] Failed to load plugin from /usr/share/metasploit-framework/plugins/Plugin_That_Does_Not_Exist.rb: cannot l

```

To start using the plugin, start issuing the commands available to us in the help menu of that specific plugin. Each cross-platform integration offers us a unique set of interactions that we can use during our assessments, so it is helpful to read up on each of these before employing them to get the most out of having them at our fingertips.

Installing new Plugins

New, more popular plugins are installed with each update of the Parrot OS distro as they are pushed out towards the public by their makers, collected in the Parrot update repo. To install new custom plugins not included in new updates of the distro, we can take the .rb file provided on the maker's page and place it in the folder at `/usr/share/metasploit-framework/plugins` with the proper permissions.

For example, let us try installing [DarkOperator's Metasploit-Plugins](#). Then, following the link above, we get a couple of Ruby (.rb) files which we can directly place in the folder mentioned above.

Downloading MSF Plugins

```

                                     Plugins
-----
dbcypn@htb[/htb]$ git clone https://github.com/darkoperator/Metasploit-Plugins
dbcypn@htb[/htb]$ ls Metasploit-Plugins

aggregator.rb      ips_filter.rb      pcap_log.rb        sqlmap.rb
alias.rb           komand.rb          pentest.rb         thread.rb
auto_add_route.rb  lab.rb            request.rb         token_adduser.rb
beholder.rb        libnotify.rb      rssfeed.rb         token_hunter.rb
db_credcollect.rb  msfd.rb           sample.rb          twitt.rb
db_tracker.rb      msgrpc.rb         session_notifier.rb wiki.rb
event_tester.rb    nessus.rb         session_tagger.rb  wmap.rb
ffautoregen.rb     nexpose.rb        socket_logger.rb   sounds.rb
growl.rb           openvas.rb

```

Here we can take the plugin `pentest.rb` as an example and copy it to `/usr/share/metasploit-framework/plugins`.

MSF - Copying Plugin to MSF

```

                                     Plugins
-----
dbcypn@htb[/htb]$ sudo cp ./Metasploit-Plugins/pentest.rb /usr/share/metasploit-framework/plugins/pentest.rb

```

Afterward, launch `msfconsole` and check the plugin's installation by running the `load` command. After the plugin has been loaded, the `help menu` at the `msfconsole` is automatically extended by additional functions

MSF - Load Plugin

```
dbcyph0n@htb[/htb]$ msfconsole -q
```

```
msf6 > load pentest
```

[illegible]

Version 1.6

```
Pentest Plugin loaded.
```

by Carlos Perez (carlos_perez[at]darkoperator.com)

```
[*] Successfully loaded plugin: pentest
```

```
msf6 > help
```

Tradecraft Commands

=====

Command	Description
-----	-----
check_footprint	Checks the possible footprint of a post module on a target system.

auto_exploit Commands

=====

Command	Description
-----	-----
show_client_side	Show matched client side exploits from data imported from vuln scanners.
vuln_exploit	Runs exploits based on data imported from vuln scanners.

Discovery Commands

=====

Command	Description
discover_db	Run discovery modules against current hosts in the database.
network_discover	Performs a port-scan and enumeration of services found for non pivot networks.
pivot_network_discover	Performs enumeration of networks available to a specified Meterpreter session.
show_session_networks	Enumerate the networks one could pivot thru Meterpreter in the active sessions.

Project Commands

=====

Command	Description
-----	-----
project	Command for managing projects.

Postauto Commands

=====

Command	Description
app_creds	Run application password collection modules against specified sessions.
get_lhost	List local IP addresses that can be used for LHOST.
multi_cmd	Run shell command against several sessions
multi_meter_cmd	Run a Meterpreter Console Command against specified sessions.
multi_meter_cmd_rc	Run resource file with Meterpreter Console Commands against specified sessions.
multi_post	Run a post module against specified sessions.
multi_post_rc	Run resource file with post modules and options against specified sessions.

```
sys_creds          run system password collection modules against specified sessions.  
  
<SNIP>
```

Many people write many different plugins for the Metasploit framework. They all have a specific purpose and can be an excellent help to save time after familiarizing ourselves with them. Check out the list of popular plugins below:

nMap (pre-installed)	NexPose (pre-installed)	Nessus (pre-installed)
Mimikatz (pre-installed V.1)	Stdapi (pre-installed)	Railgun
Priv	Incognito (pre-installed)	Darkoperator's

Mixins

The Metasploit Framework is written in Ruby, an object-oriented programming language. This plays a big part in what makes `msfconsole` excellent to use. Mixins are one of those features that, when implemented, offer a large amount of flexibility to both the creator of the script and the user.

Mixins are classes that act as methods for use by other classes without having to be the parent class of those other classes. Thus, it would be deemed inappropriate to call it inheritance but rather inclusion. They are mainly used when we:

1. Want to provide a lot of optional features for a class.
2. Want to use one particular feature for a multitude of classes.

Most of the Ruby programming language revolves around Mixins as Modules. The concept of Mixins is implemented using the word `include`, to which we pass the name of the module as a `parameter`. We can read more about mixins [here](#).

If we are just starting with Metasploit, we should not worry about the use of Mixins or their impact on our assessment. However, they are mentioned here as a note of how complex the customization of Metasploit can become.

[< Previous](#)[Next >](#)[✔ Mark Complete & Next](#)[📄 Cheat Sheet](#)

Table of Contents

Introduction

[Preface](#)[Introduction to Metasploit](#)[Introduction to MSFconsole](#)

MSF Components

[📁 Modules](#)[Targets](#)[📁 Payloads](#)[Encoders](#)[Databases](#)[Plugins & Mixins](#)



MSF Sessions

 Sessions & Jobs

 Meterpreter

Additional Features

Writing & Importing Modules

Introduction to MSFVenom

Firewall and IDS/IPS evasion

Metasploit-Framework Updates - August 2020

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left